

Exactly! 😊 Let's build directly on the confusion you just resolved.

useFormStatus — the missing piece

You already understand this:

```
const [state, formAction] = useActionState(
  analyzeWebsite,
  initialState
);
```

Think of it as:

useActionState(...)

↓

state	← result/state
formAction	← function the <form> submits to



Then:

```
<form action={formAction}>
```

When the user submits:

User clicks Submit

↓

```
<form action={formAction}>
```

↓

```
formAction()
```

↓

```
analyzeWebsite()
```

↓

```
Server Action runs
```



↓
state gets updated

Now the question is:

"While `formAction` is running, how do I know that it's currently running?"

That's exactly what `useFormStatus()` tells us.

Example

```
function SubmitButton() {  
  const { pending } = useFormStatus();  
  
  return (  
    <button disabled={pending}>  
      {pending ? "Analyzing..." : "Analyze Website"}  
    </button>  
  );  
}
```

So:

Before submit
pending = false
↓
"Analyze Website"

User submits
↓
formAction starts running
↓
pending = true
↓
"Analyzing..."
↓
formAction finishes
↓
pending = false
↓
"Analyze Website"



That's basically the whole idea. 🎯

One VERY important distinction

`useActionState` gives you the **form action function**:

```
const [state, formAction] = useActionState(...)
```

`useFormStatus` gives you the **status of the form submission**:

```
const { pending } = useFormStatus()
```

So you can mentally remember:

`useActionState` → "What happened / what is the result?"

`useFormStatus` → "Is the submission happening right now?"

And there's one little rule that matters a LOT: `useFormStatus()` **must be called by a component that is inside the** `<form>`. That's why we normally put `SubmitButton` *inside* the form.

```
<form action={formAction}>
  <input name="url" />

  <SubmitButton />
</form>
```

Now the pieces fit together nicely. 🧩



ok

👍 Perfect.

Next, we can connect `useActionState` + `useFormStatus` **together in one complete form**, so you can see exactly how the whole mutation flow works end-to-end.



ok

Awesome 🤩 Let's put the two pieces together.

🧩 `useActionState` + `useFormStatus`

Suppose our website analyzer has this Server Action:

```
"use server";

export async function analyzeWebsite(prevState, formData) {
  const url = formData.get("url");

  // do API call...
  const result = await analyze(url);

  return {
    success: true,
    result,
  };
}
```

Then our form component:

📄 JavaScript

</>

▶



```
export default function AnalyzerForm() {
  const [state, formAction] = useActionState(
    analyzeWebsite,
    { success: false, result: null }
  );

  return (
    <form action={formAction}>
      <input name="url" />

      <SubmitButton />

      {state.result && (
        <div>
          Score: {state.result.score}
        </div>
      )}
    </form>
  );
}
```

Now follow the flow carefully 📌

1. Component renders

```
useActionState(...)
  ↓
[state, formAction]
  ↓
state = initialState
formAction = special function
```



The form gets that function:

```
</> JavaScript
<form action={formAction}>
```



2. User clicks the button

```
Click "Analyze Website"
  ↓
<form> submits
  ↓
formAction runs
  ↓
analyzeWebsite runs on server
```



At this moment:

```
useFormStatus()
  ↓
pending = true
```



So the button becomes:

```
Analyzing...
```



and is disabled.

3. Server Action finishes

Suppose it returns:

```
</> JavaScript
```



```
{
  success: true,
  result: {
    score: 92
  }
}
```

That returned value becomes the new `state` .

```
analyzeWebsite()
  ↓
returns { success: true, result: ... }
  ↓
useActionState updates state
  ↓
state.result = ...
```



Meanwhile:

```
pending = false
```



So the button changes back:

```
Analyze Website
```



and the result appears.

The easiest mental model

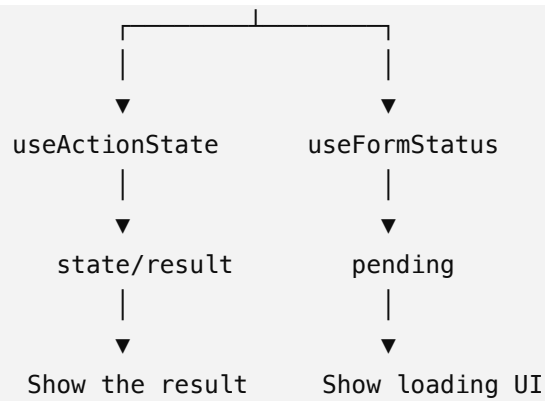
Think of the two hooks as answering **different questions**:

Hook	Question it answers
<code>useActionState</code>	"What did my Server Action return?"
<code>useFormStatus</code>	"Is my form submission currently running?"

So:

```
FORM
  |
  ▼
formAction()
  |
```





That's the complete relationship. 🔥

One final subtlety: `formAction` is the **function that triggers the action**, while `pending` is simply the **boolean status of that submission**. `useFormStatus` does **not** give you another action function. It just watches the form submission.



ChatGPT can make mistakes. Check important info.